Metrics Types and Design

the average hides the users who leave

Watch the 4-min video

In one line: use counters for things that happen, gauges for levels and histograms for latency; read latency as percentiles, not averages; measure RED for services and USE for resources; and keep labels bounded, because every label combination is a separate time series.

1 The average that lied

The dashboard says average latency is **120 ms**. But one request in a hundred takes **4 seconds**, and those users are leaving. Latency has a long tail, and an average blends the tail into the crowd. Choosing the right metric type, and reading it the right way, decides whether your graphs tell the truth.

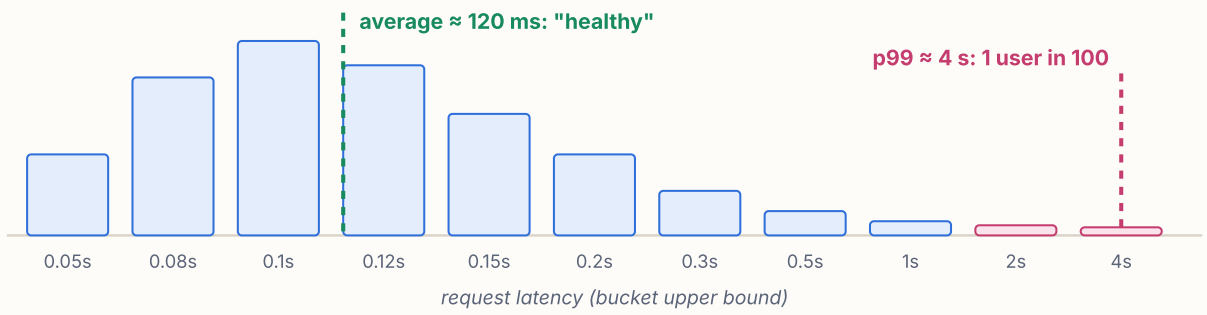


Figure 1. A long-tailed latency distribution. The mean sits in the crowd; p99 shows what the slowest 1% experience.

2 Four metric types

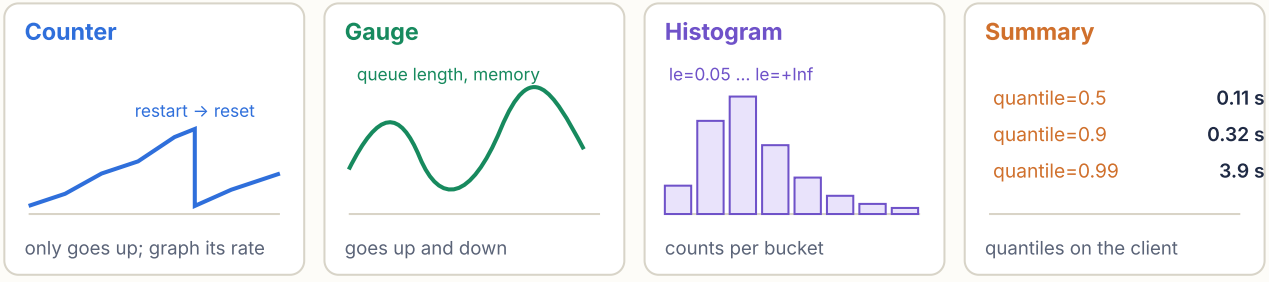


Figure 2. Counters only increase (they reset to zero on restart; `rate()` handles that). Histograms count observations per bucket; summaries ship precomputed quantiles.

Type	Use for	Read it as	Example name
Counter	events: requests, errors, bytes sent	rate per second	`http_requests_total`
Gauge	current level: memory, queue depth, connections	value now, min/max over time	`queue_messages`
Histogram	distributions: latency, payload size	percentiles from buckets	`http_request_duration_seconds`
Summary	quantiles for one instance	pre-computed p50/p99	(rarely the best choice)

3 Why histograms beat averages (and summaries)

A histogram counts observations into buckets. In Prometheus the buckets are **cumulative**: `le="0.3"` counts every request that took ≤ 0.3 s. From the buckets you estimate any percentile: p50 is the typical request, p99 the slowest one in a hundred. Crucially, bucket counts can be **summed across instances**, giving a true fleet-wide p99. A summary computes quantiles inside each process, and you **cannot average quantiles**: the mean of ten instances' p99 is not the fleet's p99. That is why histograms are usually preferred. The price: a percentile from buckets is an estimate, only as precise as the bucket boundaries near it.

4 What to measure: RED and USE

Method	For	Signals	Example
RED (Tom Wilkie)	request-driven services	**R**ate, **E**rrors, **D**uration	req/s, 5xx ratio, p99 per route
USE (Brendan Gregg)	resources: CPU, disk, pools, queues	**U**tilization, **S**aturation, **E**rrors	CPU 70%, run-queue length, disk errors

Google's "four golden signals" (latency, traffic, errors, saturation) are the same idea.

5 The cardinality problem

Each unique combination of label values is a **separate time series**, with its own memory and storage. Method × route template × status class is a few hundred series. Add `user_id` or `request_id` and you create millions: memory explodes, queries crawl, the bill jumps. And a histogram multiplies again by its bucket count (10 buckets = 10+ series per combination). Keep labels bounded; put unbounded IDs in logs and traces (lesson 59).

6 Design habits

- **Name with units**, base units: `http_request_duration_seconds`, `_bytes`, counters end in `_total`.
- **Route template, not raw path:** `/orders/{id}`, not `/orders/98123` (that is a hidden ID label).
- **Buckets around your targets:** if the SLO is 300 ms, have boundaries at e.g. 0.1, 0.3, 0.5 s.
- **Alert on symptoms** users feel (error ratio, p99), not on every internal number (lesson 67).

WORKED EXAMPLE: THE EXPORT ENDPOINT NOBODY NOTICED

A service records `http_request_duration_seconds{method, route, status_class}` with buckets 0.1 · 0.3 · 0.5 · 1 · 2.5 s. From that one histogram (PromQL in lesson 62): request rate, error ratio, and p50/p95/p99 per endpoint. `GET /orders/export` has a fine p50 but a **p99 of 8 s**; the service-wide average had hidden it for months, because export is a small share of traffic. It gets its own fix (streaming the export) and its own p99 alert.

COMMON MISTAKES

- Graphing average latency, or averaging per-instance p99s.
- Graphing a counter's raw value instead of its rate, so restarts look like drops.
- Raw URL paths or user IDs as labels: unbounded cardinality.
- Default buckets that don't bracket your SLO, so p99 can't be read accurately where it matters.

7 Key terms

Counter: monotonically increasing count; read as a rate

Gauge: value that can go up or down

Histogram: bucketed counts of observations; aggregatable

Percentile (p99): value 99% of observations are below

RED / USE: rate-errors-duration / utilization-saturation-errors

Time series: one metric name + one exact label set

8 Interview questions

Q1. Why are histograms better than averages? BASIC

They capture the whole distribution including the long tail, give percentiles like p99, and their buckets can be summed across instances (summaries can't).

Q2. What is the cardinality problem? BASIC

Every unique label combination is a separate time series, so unbounded labels like user ID or request ID explode memory, query time and cost.

Q3. Which metrics would you put on a new API service's dashboard? SCENARIO

RED per route: request rate, error ratio, p50/p99 latency from a histogram; plus USE for its resources: CPU, memory, connection-pool saturation.

Q4. Ten pods each report p99 = 200 ms via a summary. What is the fleet p99? SCENARIO

You can't tell: quantiles don't average. It could be well above 200 ms if one pod is slow; use a histogram and compute p99 over summed buckets.

Q5. Why does a counter reset not break your graphs? TRICKY

rate() and increase() detect a decrease in a counter and treat it as a reset, so the per-second rate stays correct.

Q6. Is a gauge of 'requests in the last minute' a good metric? TRICKY

No: it depends on scrape timing and can't be re-aggregated. Expose a counter and let the query compute rate over any window.

Next: SD-N-004 · Prometheus and Grafana backendroadmap.com